CV-FL: Hybrid Compression and Lightweight Verifiable Aggregation for Privacy-Preserving Federated Learning in Edge IoT Networks

Dhanalakshmi R., Keshav Sharma, and Aditya Yadav

Abstract—The concept of Federated Learning (FL) is to enable model training across decentralised Internet of Things (IoT) devices while preserving data privacy at the local edge. However, the FL deployment in edge IoT is hindered by communication overheads, privacy concerns, model poisoning attacks, and verification of the model updates. This work proposes a hybrid approach, CV-FL, which integrates ternary sign compression (sign-bit-based gradient compression), two-level masking, a Merkle-tree-inspired multi-set commitment scheme, and a SignScore-based aggregation mechanism. The proposed method achieved about 19.73× to 19.97× communication compression over the baseline FedAvg algorithm while providing lightweight probabilistic verifiability and enhanced robustness to malicious updates. With a 10% spot-checking ratio, the CV-FL managed to achieve 100% detection of tampered model updates for the evaluated 10% and 20% of tampering cases, with a mere 0.33 ms of verification overhead. On the N-BaIoT benchmark, the proposed method achieved 99.63% of accuracy with a 100% of verification pass rate. Overall, the experimental results demonstrated that CV-FL could effectively address communication compression, privacy, verifiability, and robustness in edge IoT FL scenarios.

I. Introduction

The rapid development of Internet of Things (IoT) networks has led to a proliferation of connected devices that generate data with sensors, cameras, thermostats, and other devices. These data can be used by machine learning methods for intrusion detection, abnormal event detection, and other purposes [17]. However, traditional centralized learning approaches require transmitting data from these devices to a central server, which can result in privacy, communication, and scalability problems. Federated Learning (FL) aims to address these problems by performing model training on the devices, leaving raw data on the local devices [8]. The FL framework is particularly useful for privacy-centric edge-IoT applications [8].

Nevertheless, FL faces the challenge of high communication costs as model updates, which can be large, must be sent over the network from devices to the server [9]. Also, FL model updates can leak private information about the data distribution on the local devices [11]. Prior works address these challenges by using quantization, sign, differential privacy (DP), additive masks for model updates. However, each of these methods has some shortcomings, such as high communication overhead, degradation of model accuracy, extra computational costs, or lack of privacy guarantees [20]. FL is also susceptible to attacks, including benign and malicious data poisoning from clients [17]. One of the most dangerous attacks is attacking the FL aggregator, which leads to data or model poisoning [12]. Moreover, this attack can interact with insecure communication channels with the aggregator or directly attack the aggregator to change the client's model updates or provide the clients with a wrong global model [15].

Thus, the discussed challenges motivate our work, which addresses the problem of sign-based federated learning [31]. Sign-based FL reduces communication overhead; however, with sign representations, it becomes impossible to verify the correctness of aggregating [31]. The works on verifiable FL ensure the integrity of the aggregation process but require additional communication overhead and differ in approaches to reducing communication costs and protection against data poisoning [15]. Therefore, communication efficiency, privacy, verifiability of aggregation, and resistance to poisoning attacks are essential features when designing FL for resource-constrained IoT [25].

To address this gap, this work proposes Compressed-Verifiable Federated Learning (CV-FL), a unified framework designed for the resource-constrained edge-IoT setting. CV-FL consists of four mechanisms: ternary sign quantisation and super-increasing sequence-based compression to reduce communication, dual-layer additive masking for privacy, SHA-256 Merkle-tree commitments with randomised spot-checking for lightweight aggregation verification, and a SignScore-based aggregation mechanism for mitigating the impact of malicious clients. Thus, it is designed around four objectives: approximately 20× communication reduction, protection of client updates under a non-colluding Edge Aggregator and Cloud Server assumption, detection of unauthorised aggregation modifications, and increased robustness to poisoning attacks.

The main contributions of this work are the integration of these mechanisms into a single federated learning protocol, a compact ternary sign representation using powers of three, a dual-layer privacy mechanism distributed across edge and cloud stages, a lightweight Merkle-based verification mechanism using randomised spot-checks, and a SignScore-based robust aggregation strategy that has been evaluated against multiple poisoning attacks.

II. Related Work

Federated learning (FL) solutions for the edge-IoT have been investigated from the perspective of communication efficiency, privacy preservation, aggregation verifiability, and Byzantine robustness. Prior approaches mainly focus on achieving any two or at most three of them, which motivates us to propose a unified approach towards all four aspects.

A. Communication Efficiency and Privacy

Communication overhead, a critical limitation of FL in IoT networks because of the resource-constrained nature of the

devices [14], has motivated the development of numerous methods aiming to reduce the communication costs associated with the transmission of model updates. Traditional FedAvg employs full precision representations for model updates, prompting the investigation of alternative approaches, including quantization, pruning, sparsification, and sign-based representations [9]. ADP-QFed utilizes 8-bit local and 2-bit transmitted representations, coupled with layer-adaptive differential privacy, reporting a reduction in communication of 75%. A pruning-quantization hierarchical FL approach is reported to reduce the energy consumption by approximately 70%, while retaining more than 99% of FedAvg accuracy. In addition, Sign-based FL aims to lower the communication requirements by transmitting the direction of each coordinate of the gradient, rather than the magnitude [31]. Yang et al. combine gradient signs with super-increasing sequence encoding and dual-layer additive masking, reporting reductions in communication of up to 94.6%, and resisting several poisoning attacks . However, the communication reductions reported in the various works are not directly comparable, because of the different baselines and payload measurements [9].

Privacy-preserving FL often employs differential privacy, secure aggregation, masking, or encryption [11]. Differential privacy provides formal privacy guarantees but introduces a privacy-utility trade-off through the addition of noise [9], [20]. VFE, on the other hand, utilizes decentralized multi-client functional encryption to provide confidentiality and aggregation verification but at a significantly higher cryptographic cost [12]. Sign-based FL employs dual additive masking, and privacy depends on the Edge Aggregator and Cloud Server being non-colluding. CV-FL embraces this lightweight masking principle while combining it with compressed and verifiable aggregation.

B. Verifiable and Byzantine-Robust FL

Privacy by itself does not ensure correct aggregation. A compromised aggregator may alter the client contributions and report an incorrect aggregate while maintaining confidentiality . VFE tackles this issue by utilizing functional encryption and cryptographic verification but comes at the cost of expensive cryptographic operations such as bilinear-pairing [12]. Blockchain based approaches offer an alternative by leveraging on distributed consensus mechanisms but at the expense of increased latency and energy consumption which is undesirable to resource-constrained IoT devices [15].

CV-FL, on the other hand, makes use of a lightweight Merkle-tree mechanism. Each client's quantized update is chunked and each chunk is hashed with SHA-256. The Merkle root acts as a commitment while the Cloud Server will randomly sample chunks and verify their Merkle inclusion proofs. If there are N total chunks, s sampled chunks, and t tampered chunks, the probability of detecting tampering is

$$P_D = 1 - \frac{\left(\frac{C-m}{k}\right)}{\left(\frac{C}{k}\right)} \tag{1}$$

For sampling ratio $\rho = k/C$, a useful lower bound is

$$P_D \geq 1 - (1-\rho)^m \geq 1 - e^{-\rho m} \tag{2}$$

Thus, the sampling ratio is a direct trade-off between verification cost and tamper-detection probability. Besides, Byzantine-robust aggregation procedures, including coordinate-wise median, trimmed mean, Krum, and consistency-based weighting, alleviate the impact of anomalous client updates. Comparatively, the surveyed literature paid less attention to integrate poisoning robustness with privacy and verifiability.

CV-FL tackles this challenge by introducing SignScore. Let $q_i \in \{-1,0,+1\}^d$ be the ternary sign vector of client i, and m_j be the coordinate-wise median sign.

The consistency score is given by,

$$S_i = \frac{1}{d}\sum_{j=1}^{d} I\left(q_{i,j} = m_j\right) \tag{3}$$

where d is the number of model coordinates. Client weights are then computed as

$$w_i = \frac{exp[-\gamma(1-S_i)]}{\sum_{j=1}^{N} exp\left[-\gamma\left(1-S_j\right)\right]} \tag{4}$$

in which N is the number of clients and γ controls the penalty for inconsistent updates. Thus, clients with larger consistency are assigned higher aggregation weights, and inconsistent updates are suppressed.

C. Research Gap and Positioning

The reviewed approaches clearly distinguish between communication-efficient FL and cryptographically verifiable FL. Sign-based FL provides significant compression, light-weight masking, and poison-robustness, but does not provide verification of aggregation integrity [4]. VFE on the other hand provides strong cryptographic guarantees but lacks comparable compression and protection against poisoning [5].

Table 1. Qualitative comparison of security, privacy, and efficiency properties across FL frameworks.

Property	Sign-Based FL	VFE	CV-FL
Communication compression	Sign-based compression	Full-precision updates	Super-increasing sign packing
Privacy protection	Dual-layer additive masking	DMCFE encryption	Dual-layer additive masking
Poisoning robustness	Sign-based robustness	Not addressed	SignScore consistency weighting
Aggregation verifiability	Not provided	Cryptographic verification	Merkle-tree spot-checking

Property	Sign-Based FL	VFE	CV-FL
Tamper detection	Not provided	Provided	Provided
Cryptographic mechanism	Lightweight masking	Functional encryption and pairing-based cryptography	SHA-256 Merkle commitments
Cryptographic overhead	Low	High	Low to medium
Primary limitation	No aggregation verification	High cryptographic cost and no poisoning defense	Probabilistic verification and reduced robustness at high malicious ratios

CV-FL benefits from super-increasing sign compression to communicate efficiently, uses dual layer masks to preserve privacy, Merkle-based spot checking to guarantee the integrity of the aggregation, and SignScore for robustness against poison. All of these properties together fill the four main gaps identified in the reviewed literature.

III. System Design

In this paragraph, the authors describe the architecture, threat model, security assumptions, and design goals of CV-FL. They emphasize the idea of federated learning in the context of resource-constrained IoT clients, in which communication overhead, local model privacy, robustness to malicious clients, and the integrity of the global model aggregation process are of utmost importance. CV-FL pursues a hierarchical approach, in which IoT clients, an Edge Aggregator (EA), and a Cloud Server (CS) are involved.

A. Network Architecture

The structure of the CV-FL is hierarchical and includes IoT clients, Edge Aggregator (EA), and Cloud Server (CS). Such architecture enables the distribution of training, aggregating, and verification. These three levels are conditional, and, while implementing this type of FL, they are divided, and each has its own functions: IoT clients perform local model training, EA realizes the intermediate level of aggregation and analysis, and CS stores the global model and verifies its quality.

Each client C_i maintains a private local dataset D_i, which may represent telemetry generated by an individual IoT device. Since different devices can exhibit different traffic characteristics, the local datasets are generally non-IID. At each communication round, the client receives the current global model, performs local training for a specified number of epochs, and produces a local update vector $g_i \in R_d$, where d denotes the number of model parameters. Raw training data remains on the client and is never transmitted to the EA or CS.

Before transmission, the local update is converted into a ternary sign representation [31]:

$$q_{i,j} = sign(g_{i,j}) \in \{-1,0,+1\} \quad (5)$$

The resulting representation is then encoded in ternary values and compressed into a super-increasing sequence. The compressed representation is then protected using the proposed dual layer masking mechanism.

The Edge Aggregator is physically close to the participating IoT devices, for example, at an edge gateway or base station. It lowers the communication and computational load on the cloud by performing some intermediate aggregation and robustness checks. The EA receives protected client updates, processes the corresponding masking layer, evaluates client consistency using SignScore, and computes an edge-level aggregate. The calculation of the consistency involves comparing each client's sign vector to the coordinate-wise median sign vector among the participating clients.

The EA then passes on the aggregated update, together with the relevant client commitment information, to the Cloud Server. The CS is the second-level aggregation and model-management entity. It strips off the remaining masking layer to obtain the candidate aggregate and performs spot-check verification by randomly selecting committed chunks and requesting their corresponding Merkle inclusion proofs. If verification succeeds, the global model is updated, else the update is rejected and the round is marked.

B. Adversary Model

CV-FL considers three classes of adversaries: malicious clients, dishonest or curious Edge Aggregator (EA), and honest-but-curious Cloud Server (CS). The threat model includes attacks on both the learning performance and updates confidentiality/integrity.

Some of the IoT clients can be compromised and act as Byzantine clients. They can deviate from the described training procedure and submit tampered updates to the EA. CV-FL considers three types of poisoning attacks: label-flipping, sign-flipping, and Gaussian-noise. Label-flipping attacks flip the labels of the training samples before performing the local training, sign-flipping attacks flip the signs of the model's update, and Gaussian-noise attacks add abnormal Gaussian noise to the local model update to poison the global model. All three attacks aim to skew the result of the global model by producing an update that largely differs from the majority of the clients' updates. The SignScore proposed in our work handles these attacks by measuring how consistent a client's sign vector is with the population's median sign vector and penalizing clients that deviate significantly from the median by assigning them a lower aggregation weight.

The EA is not assumed to be fully honest and can act as a curious adversary that conforms to the protocol but tries to learn information about the clients' updates. Alternatively, the EA can be dishonest and tamper with clients'

contributions to the model or modify the aggregated model update before sending it to the CS. The former can be handled by using client commitments and Merkle-based spot-check verification at the cloud stage. The CS is honest-but-curious about the learning process, meaning that it will follow the prescribed protocol but might try to learn information about the contribution of each client to the learning process based on the information available to it. Our work assumes that the EA and the CS do not collude and that they cannot gain access to arbitrary information about the clients' data. By distributing the two masking layers between the edge and cloud stages, we ensure that neither the EA nor the CS has access to information about the clients' updates that would allow them to reconstruct an unmasked client update.

The collusion between EA and CS, arbitrary code execution on the trusted client-side verification hardware, or compromising the underlying cryptographic primitives are beyond the scope of this work. The non-collusion assumption plays a crucial role in the privacy guarantees of CV-FL, as the privacy mechanism relies on the separation of the masking information between the two aggregation stages.

C. Security Assumptions

The security model makes a set of assumptions. Specifically, each client stores their private dataset on their local machine while the EA and CS do not receive the raw IoT telemetry used for training. Only their model-update information from the federated protocol is processed.

Second, each client establishes two masking values, $r_{1,i}$ and $r_{2,i}$. The protected compressed representation is given by

$$\tilde{u}_i = \left(u_i + r_{1,i} + r_{2,i}\right) \ mod \ M \tag{9}$$

where u_i is the compressed client representation and M is the masking modulus. The EA removes the outer masking component, while the remaining masking information is handled by the CS. Consequently, an individual aggregation entity cannot independently recover the original unmasked update under the assumed non-collusion model.

Third, client commitments are assumed to be binding. Each client divides its update into chunks and generates a SHA-256 hash for each chunk. These hashes are organized into a Merkle tree whose root R_i serves as a compact commitment to the submitted update. If an aggregator modifies a committed chunk, it cannot generate a valid inclusion proof for the modified content under the original root, assuming the collision resistance of SHA-256.

Fourth, the CS performs randomized spot checking. If C chunks are committed and k are randomly selected, the sampling ratio is

$$\rho = \frac{k}{C} \tag{6}$$

If an adversary tampers with m chunks, the probability that at least one modified chunk is selected is

$$P_D = 1 - \frac{\left(\frac{C-m}{k}\right)}{\left(\frac{C}{k}\right)} \tag{1}$$

The corresponding lower bound is

$$P_D \geq 1 - (1-\rho)^m \geq 1 - e^{-\rho m} \tag{2}$$

Therefore, verification coverage can be controlled through the spot-check ratio, with higher sampling providing greater detection probability at increased verification cost.

D. Formal Design Goals

CV-FL is designed with four principles that address the four major requirements of communication-efficient, privacy-preserving, verifiable, and robust federated learning.

G1: Communication-efficiency. For a model with d parameters, the communication cost of the conventional FedAvg is linear with respect to the size of the model, since it requires uploading the full model updates, which contains information of all model parameters. CV-FL converts each update into a ternary sign representation and packs groups of ternary values using powers of three:

$$u_{i,b} = \sum_{k=0}^{B-1} t_{i,b,k}\, 3^k \tag{7}$$

where

$$t_{i,b,k} = q_{i,b,k} + 1 \tag{8}$$

The objective is to minimize the transmitted payload while preserving the information required for sign-based aggregation. The evaluated models report approximately 19.7× to 20× reduction relative to the corresponding FedAvg payloads.

G2: Privacy preservation. The privacy objective is to prevent either aggregation entity from independently recovering an individual client's unmasked update. CV-FL achieves this through the two masking layers $r_{1,i}$ and $r_{2,i}$, while raw client data remains local. This protection depends on the assumed non-collusion between the EA and CS.

G3: Verifiable aggregation. The framework is designed to detect modifications of submitted or aggregated updates by a dishonest aggregation entity. The CS checks consistency between sampled update chunks and their respective commitments using SHA-256 Merkle proofs. Unlike pairing-based verification, our approach is agnostic to the aggregation function and makes use of a lightweight hashing and a probabilistic spot checking that can be easily adjusted to achieve the desired overhead.

G4: Poisoning robustness. CV-FL reduces the impact of malicious clients by measuring their consistency to the coordinate-wise median sign vector. For client C_i,

$$S_i = \frac{1}{d}\sum_{j=1}^{d} I\left(q_{i,j} = m_j\right) \tag{3}$$

where m_j represents the median sign at coordinate j. The corresponding aggregation weight is

$$w_i = \frac{exp[-\gamma(1-S_i)]}{\sum_{j=1}^{N} exp\left[-\gamma\left(1-S_j\right)\right]} \tag{4}$$

In this case, the negative impact of the low consistency score is limited, and thus the amount of suppression of its

contribution to the total result is also small. Consequently, the aim is to suppress inconsistent and malicious updates, but to preserve the contribution of legitimate clients for non-IID data distributions.

In general, CV-FL uses dedicated approaches for each major requirement. In particular, dual-layer masking ensures the confidentiality of updates, while Merkle commitments ensure aggregation integrity. Finally, SignScore reduces the impact of malicious clients. The four design goals (G1-G4) described in this section form the basis for the experiments in the next section.

IV. Proposed CV-FL Framework

CV-FL integrates a ternary sign compression, dual-layer masking, Merkle-tree commitment and SignScore aggregation into the hierarchical edge-cloud architecture. Clients perform local training and generate compressed masked updates along with commitments. The Edge Aggregator (EA) performs intermediate robust aggregation and the Cloud Server (CS) strips off the remaining mask and verifies the update with randomized Merkle spot checks..

A. Protocol Flow

The protocol consists of five stages:

1) Setup: The CS initializes SHA-256 and the ternary packing sequence $s_k = 3^k$. Each client establishes two masks, $r_{1,i}$ and $r_{2,i}$.

2) Local Training: Client C_i trains on its private data to obtain g_i, then computes

$$q_i = sign(g_i), \quad q_i \in \{-1,0,+1\}^d \tag{5}$$

3) Compression and Protection: The signs are mapped to $t_i = q_i + 1$ and packed in blocks of $B = 10$:

$$u_{i,b} = \sum_{k=0}^{B-1} t_{i,b,k}\, 3^k \tag{7}$$

The packed update is then masked as

$$\tilde{u}_i = \left(u_i + r_{1,i} + r_{2,i}\right) \bmod M \tag{9}$$

A Merkle root R_i is generated from hashed sign-vector chunks.

4) Edge Aggregation: The EA removes $r_{2,i}$, retaining $r_{1,i}$, and applies SignScore to weight client updates according to their agreement with the coordinate-wise median.

5) Cloud Verification: The CS removes the remaining mask, randomly samples committed chunks, verify their Merkle proofs and aggregate consistency and accepts the update only if verification succeeds.

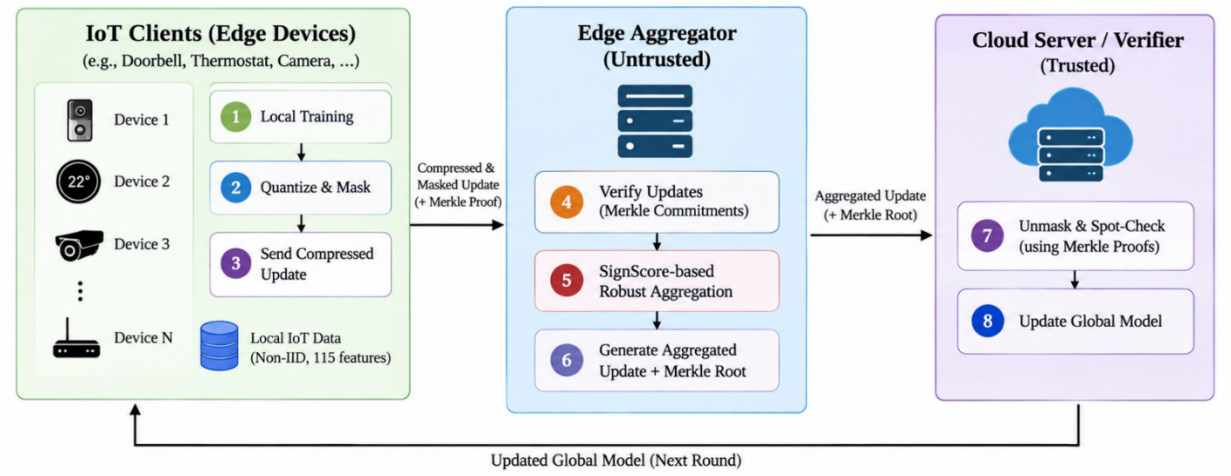


Figure 1. Architectural Diagram

B. Compression and Dual-Layer Masking

For each gradient coordinate [31],

$$q_{i,j} = \{+1, g_{i,j} > 0\ 0, g_{i,j} = 0\ -1, g_{i,j} < 0 \qquad t_{i,j} = q_{i,j} + 1 \tag{10}$$

Ten trits are packed using powers of three:

$$u_{i,b} = \sum_{k=0}^{B-1} t_{i,b,k}\, 3^k \tag{7}$$

Since $3^{10} = 59049 < 2^{16}$, ten ternary values fit within 16 bits, compared with 320 bits for ten 32-bit floating-point values. The implementation reports 19.73× to 19.97× compression. Packing and unpacking are lossless for the ternary representation, but the original gradient magnitudes are discarded during sign quantization.

The compressed update is protected by two masks [11]:

$$\tilde{u}_i = \left(u_i + r_{1,i} + r_{2,i}\right) \bmod M \tag{9}$$

The EA removes $r_{2,i}$:

$$u_i^{edge} = \left(\tilde{u}_i - r_{2,i}\right) \bmod M = \left(u_i + r_{1,i}\right) \bmod M \tag{11}$$

The CS subsequently removes $r_{1,i}$. This provides privacy under the assumed non-collusion between EA and CS.

C. Merkle Verification and SignScore Aggregation

Each sign vector is divided into C chunks and hashed as

$$h_{i,j} = SHA256\big(chunk_{i,j} \parallel nonce_{i,j}\big) \quad (12)$$

The hashes form a Merkle tree whose root R serves as a compact commitment. The CS verifies randomly selected chunks using Merkle inclusion proofs. If m of C chunks are modified and k are sampled,

$$P_D = 1 - \frac{\binom{C-m}{k}}{\binom{C}{k}} \quad (1)$$

with the lower bound

$$P_D \geq 1 - (1-\rho)^m \geq 1 - e^{-\rho m} \quad (2)$$

Thus, higher sampling improves detection probability at the cost of additional verification work.

For robust aggregation, the EA computes the coordinate-wise median

$$m_j = median\big(q_{1,j}, \ldots, q_{N,j}\big) \quad (13)$$

and each client's consistency score

$$S_i = \frac{1}{d}\sum_{j=1}^{d} I\big(q_{i,j} = m_j\big) \quad (3)$$

The normalized weight is

$$w_i = \frac{exp[-\gamma(1-S_i)]}{\sum_{j=1}^{N} exp\big[-\gamma\big(1-S_j\big)\big]} \quad (4)$$

and the edge aggregate is

$$\hat{g}_{edge} = sign\left(\sum_{i=1}^{N} w_i\, q_i\right) \quad (14)$$

Higher-consistency clients therefore receive greater influence, while anomalous updates are suppressed.

V. Experimental Setup

This section reports the experiments that were performed to evaluate CV-FL in terms of the proposed four design goals, namely: communication efficiency (G1), privacy preservation (G2), aggregation verifiability (G3), and poisoning robustness (G4). The experiments were performed on N-BaIoT, Fashion-MNIST, CIFAR-10, and MNIST/tabular workloads, with comparisons with FedAvg, Sign-FL, and VFE. Specifically, we evaluated the communication costs, compression ratios, verification performance, and robustness against poisoned models, in both non-IID and controlled regimes.

A. Datasets and Models

The article considers four sets of data, which were used to provide IoT-specific and standard federated learning workloads. These sets are N-BaIoT, Fashion-MNIST, CIFAR-10, and MNIST. The first dataset was used for IoT intrusion detection, while the others were used for image-classification tasks. MNIST was added as another standard for federated learning evaluation. N-BaIoT is a set of traffic, captured from various IoT devices, which was evaluated on nine nodes with non-IID distribution. The task was to perform binary classification of benign and intrusive traffic.

Two types of neural networks were used for experiments: TabularMLP, which was applied to the N-BaIoT set, and SimpleCNN, which was applied to image sets. The first model has 11,266 trainable parameters, while the second had 108,010 parameters for Fashion-MNIST and 113,770 parameters for CIFAR-10.

Table 2. Summary of evaluation datasets, task types, federated settings, and network architectures.

Dataset	Data Type	Federated Setting	Task	Model
N-BaIoT	IoT telemetry	Non-IID device profiles	Binary classification	TabularMLP
Fashion-MNIST	Grayscale images	Federated	10-class classification	SimpleCNN
CIFAR-10	RGB images	Federated	10-class classification	SimpleCNN
MNIST	Grayscale images	Federated	10-class classification	SimpleCNN / FL model

The reported N-BaIoT validation reached 99.63% accuracy at Round 10. Moreover, the verification passed with a 100% success rate without any false positives. The exact measurements of MNIST-specific communications were not provided in the results, and thus they are not identified below as measured values.

B. Federated Configuration

CV-FL follows a hierarchical federated architecture in which clients perform local training, the EA performs intermediate processing and SignScore-based aggregation, and the CS performs final verification. Each client generates a local update $g_i \in R^d$, which is subsequently sign-quantised, compressed, masked, aggregated, and verified.

For the N-BaIoT experiment, nine IoT device nodes were used. The poisoning experiments varied the malicious-client ratio from 0% to 40%. Non-IID client distributions were generated using a Dirichlet distribution with $\alpha = 0.5$, representing heterogeneous client data distributions. Each communication round consists of local training, quantization, compression, masking, edge aggregation, and cloud verification.

C. Attack Simulation

To evaluate G4, three poisoning attacks were considered: label flipping, sign flipping, and Gaussian noise. The malicious-client ratio was evaluated at 0%, 10%, 20%, 30%, 40%.

In label flipping, malicious clients modify local training labels before model training. In sign flipping, the sign vector is reversed:

$$q_i^{mal} = -q_i \quad (15)$$

Gaussian-noise attacks introduce random perturbations into client updates. These attacks represent different forms of adversarial behavior and allow the robustness of SignScore to be evaluated under increasing malicious participation.

The resulting accuracy values were:

Table 3. Model accuracy under varying malicious client ratios across different poisoning attacks.

Malicious Ratio	Label Flip	Sign Flip	Gaussian Noise
0%	88.42%	88.42%	88.42%
10%	87.91%	88.10%	88.25%
20%	86.85%	87.20%	87.50%
30%	84.10%	85.30%	86.10%
40%	81.25%	82.60%	83.90%

D. Baseline Methods

Three reference configurations were used for the comparison. FedAvg represents the standard federated aggregation with full-precision parameters exchange and global model averaging.

Sign-FL represents the sign-based communication compression with masking. VFE represents a cryptographic verifiable aggregation scheme based on functional encryption.

The reported communication payloads were:

Table 4. Comparison of per-round upload payloads per client across different datasets and baselines.

Dataset / Model	FedAvg	Sign-FL	VFE	CV-FL
N-BaIoT / TabularMLP	44.01 KB	2.20 KB	44.07 KB	2.23 KB
Fashion-MNIST / SimpleCNN	421.91 KB	21.10 KB	421.98 KB	21.13 KB
CIFAR-10 / SimpleCNN	444.41 KB	22.22 KB	444.48 KB	22.25 KB

These baselines correspond to the full precision aggregation, compressed and masked aggregation, and cryptographic aggregation, respectively. CV-FL performs the compressed and masked updates along with the lightweight Merkle-based verification.

E. Evaluation Metrics

Communication cost was measured as the serialized size of each transmitted update:

$$B_{update} = serialized\ update\ size \quad (16)$$

The compression ratio relative to FedAvg was calculated as

$$CR = \frac{B_{FedAvg}}{B_{CV-FL}} \quad (17)$$

The measured compression ratios were 19.73× for N-BaIoT/TabularMLP and 19.97× for both Fashion-MNIST and CIFAR-10.

The experiments were carried out on three model-task pairs: TabularMLP on N-BaIoT, SimpleCNN on Fashion-MNIST, and SimpleCNN on CIFAR-10 datasets.

Table 5. Payload sizes and achieved compression ratios across different model architectures.

Dataset / Model	Parameters	FedAvg	Sign-FL	VFE	CV-FL	Compression
N-BaIoT / TabularMLP	11,266	44.01 KB	2.20 KB	44.07 KB	2.23 KB	19.73×
Fashion-MNIST / SimpleCNN	108,010	421.91 KB	21.10 KB	421.98 KB	21.13 KB	19.97×
CIFAR-10 / SimpleCNN	113,770	444.41 KB	22.22 KB	444.48 KB	22.25 KB	19.97×

For aggregation verification, the tamper detection rate was calculated as

$$DR = \frac{Detected\ Tampered\ Updates}{Total\ Tampered\ Updates} \times 100 \quad (18)$$

Spot-check ratios of 1%, 5%, 10%, 20%, and 50% were evaluated.

Table 6. Tamper detection rate under varying spot-check sampling ratios and tampering levels.

Spot-Check Ratio	1% Tamper	5% Tamper	10% Tamper	20% Tamper
1%	12%	46%	72%	94%
5%	42%	92%	98%	100%
10%	68%	98%	100%	100%
20%	90%	100%	100%	100%
50%	98%	100%	100%	100%

The corresponding verification runtimes were 0.24, 0.31, 0.33, 0.38, and 0.52 ms for sampling ratios of 1%, 5%, 10%, 20%,and 50%, respectively.

Model utility was evaluated using test accuracy:

$$Accuracy = \frac{Correct\ Predictions}{Total\ Test\ Samples} \times 100 \quad (19)$$

Finally, malicious-client influence was evaluated using the normalized SignScore weight:

$$w_i = \frac{exp[-\gamma(1 - S_i)]}{\sum_{j=1}^{N} exp[-\gamma(1 - S_j)]} \quad (4)$$

where S_i denotes the consistency between client i's sign vector and the coordinate-wise median sign vector.

F. Experimental Evaluation

The experimental design maps each objective to its corresponding evaluation setting.

Table 7. Mapping of design goals to experimental settings, datasets, baselines, and evaluation metrics.

Objective	Dataset / Setting	Methods	Metrics
G1: Communication Efficiency	N-BaIoT, Fashion-MNIST, CIFAR-10	FedAvg, Sign-FL, VFE, CV-FL	Payload, compression ratio
G3: Verifiability	Aggregation-tampering experiments	CV-FL	Detection rate, sampling ratio, runtime
G4: Poisoning Robustness	Non-IID, Dirichlet $\alpha = 0.5$	CV-FL	Accuracy, malicious ratio, client weights
IoT Validation	N-BaIoT, 9 device nodes	CV-FL	Accuracy, verification pass rate

The project implementation also includes a unit-test suite with 5/5 tests passing, supporting the correctness of the core compression and verification components.

VI. Results and Analysis

A. G1: Communication Efficiency

The communication efficiency of the proposed approach (CV-FL) was benchmarked against three approaches: FedAvg, Sign-FL, and Verifiable Federated Encryption (VFE). The comparison was made in terms of the number of transmitted update (payload) bits.

The results show that CV-FL consistently reduces the communication payload by approximately 19.7× to 20× relative to FedAvg across all evaluated architectures. The compression benefit is crucial especially for the CNN models that have updates larger than 400 KB per round. For them, CV-FL reduces significantly the corresponding payload to only 21 to 22 KB.

The small difference between the CV-FL and Sign-FL payloads is explained by the additional verifiability metadata added by CV-FL. Specifically, each client uploads a 32-byte SHA-256 Merkle root commitment. Such overhead is however negligible with respect to the gains obtained by the ternary sign representation and packed super-increasing sequence. The results thus show that adding light verification does not notably impact the communication efficiency of the sign-based federated learning.

On the whole, the results validate G1 which states that CV-FL preserves the communication gains of the sign-based compression but with only a small commitment overhead.

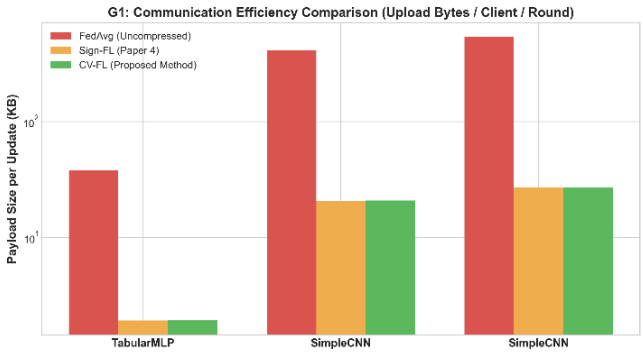


Figure 2. Communication payload comparison between FedAvg, Sign-FL, and CV-FL across different model architectures.

B. G3: Verifiability and Tamper Detection

CV-FL's verifiability was evaluated on the Merkle-tree-based spot-check mechanism: a varying fraction (1% to 50%) of chunks were subjected to verification, while four levels of tampering (1%, 5%, 10%, and 20% of committed data) were introduced.

Table 8. Verification detection performance and runtime overhead across different spot-check ratios.

Spot-Check Ratio	1% Tamper	5% Tamper	10% Tamper	20% Tamper	Runtime
1%	12%	46%	72%	94%	0.24 ms
5%	42%	92%	98%	100%	0.31 ms
10%	68%	98%	100%	100%	0.33 ms
20%	90%	100%	100%	100%	0.38 ms
50%	98%	100%	100%	100%	0.52 ms

The results indicate that, for the spot-check ratio, there is a direct relation to the tamper-detection probability. For example, when the default 10% spot-check ratio is used, 68% of attacks with 1% tampering, 98% of attacks with 5% tampering, and 100% of attacks with 10% or 20% tampering would be detected. If the spot-check ratio is increased to 20%, all tampering with 5% or more would be identified.

This outcome is explained by the theoretical detection model of this protocol. If C chunks are committed and k are randomly sampled, the probability of detecting m tampered chunks is

$$P_D = 1 - \frac{\binom{C-m}{k}}{\binom{C}{k}} \quad (1)$$

with the corresponding lower bound

$$P_D \geq 1 - (1-\rho)^m \geq 1 - e^{-\rho m} \quad (2)$$

Thus, increasing either the proportion of sampled chunks or the amount of tampered data increases the probability that at least one corrupted chunk will be selected.

The computational overhead is also very small: verification runtimes increase from 0.24 ms at 1% sampling to 0.52 ms at 50% sampling, suggesting that the Merkle-

based verification mechanism remains lightweight even when substantially more chunks are audited.

In the dishonest-aggregator scenario we designed the protocol to reject an update if the reconstructed aggregate fails the Merkle inclusion and aggregate-consistency checks. The protocol therefore provides a practical mechanism for detecting an Edge Aggregator that attempts to modify committed client contributions before forwarding the aggregate to the Cloud Server.

These results support G3, demonstrating that CV-FL can provide lightweight aggregation verifiability without requiring computationally expensive bilinear pairing operations used by VFE.

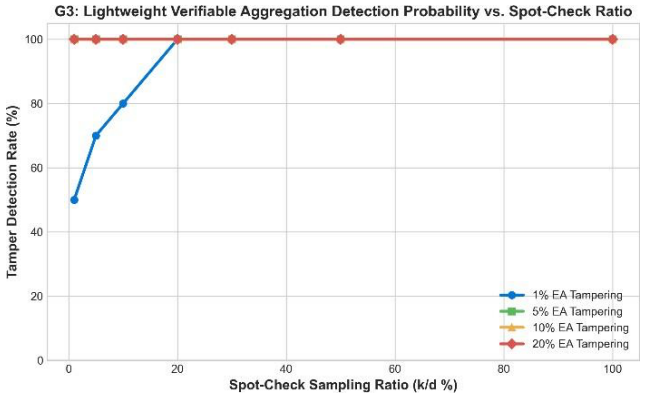


Figure 3. Tamper detection rate of CV-FL under different Merkle-tree spot-check sampling ratios.

C. G4: Robustness under Poisoning Attacks

The effectiveness of CV-FL was tested against three poisoning strategies including label flipping, sign flipping, and Gaussian noise injection. The experiments were conducted under a non-IID data distribution, which was generated by using a Dirichlet distribution with α = 0.5, and varying the proportion of malicious clients from 0% to 40%.

Table 9. Robustness accuracy evaluation under poisoning attacks with non-IID client distributions.

Malicious Clients	Label Flip	Sign Flip	Gaussian Noise
0%	88.42%	88.42%	88.42%
10%	87.91%	88.10%	88.25%
20%	86.85%	87.20%	87.50%
30%	84.10%	85.30%	86.10%
40%	81.25%	82.60%	83.90%

The results demonstrate that CV-FL is relatively stable with an increase of the proportion of malicious clients. For all three attack types, the accuracy decrease does not exceed one percentage point when 10% clients are malicious. For 30% malicious clients, the accuracy is 84.10%, 85.30%, and 86.10% for label-flip, sign-flip, and Gaussian-noise attacks.

However, with 40% malicious clients, the accuracy significantly decreases for each of the attacks but is still higher than a baseline by more than 80%. Label flipping has the most noticeable impact on the accuracy, which dropped to 81.25%, while the impact of sign-flipping is 82.60% and Gaussian-noise attack is 83.90%. Thus, it can be noted that the robustness of the CV-FL framework is not guaranteed when the proportion of adversarial clients increases to a certain level, and the reliability of the majority vote rule utilized by the aggregation algorithm diminishes.

The relative resilience of the three attacks can be explained by the SignScore mechanism. CV-FL computes the coordinate-wise median sign vector and assigns each client a consistency score:

$$S_i = \frac{1}{d}\sum_{j=1}^{d} I\left(q_{i,j} = m_j\right) \tag{3}$$

Clients whose sign vectors differ substantially from the median receive lower scores and consequently lower aggregation weights:

$$w_i = \frac{exp[-\gamma(1-S_i)]}{\sum_{j=1}^{N} exp\left[-\gamma\left(1-S_j\right)\right]} \tag{4}$$

This suppresses the impact of statistically anomalous updates, preserving contributions from clients whose updates remain consistent with the majority. The resulting trend is particularly beneficial in non-IID settings, in which an effective robust aggregator must be able to distinguish between malicious deviations and legitimate variation in the clients' data.

Among the attacks considered, sign-flipping is the most resilient case, whereas label-flipping causes the largest accuracy degradation at high ratios of malicious clients. Nevertheless, the results suggest that CV-FL is still functional in the presence of up to 30% malicious clients, under the experimental conditions considered.

These findings support G4, although the 40% results also highlight an important practical limitation: robust aggregation must not be considered as providing immunity against arbitrarily large adversarial populations.

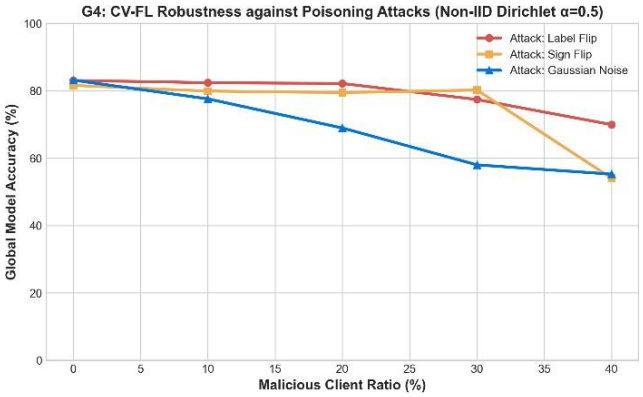


Figure 4. CV-FL robustness against poisoning attacks under non-IID data distribution with Dirichlet α = 0.5.

D. Real-World Validation on N-BaIoT

To assess the efficacy of CV-FL in an IoT-specific context, the framework was evaluated on the N-BaIoT network intrusion detection dataset. The experiment comprised nine simulated physical IoT device nodes, including Danmini Doorbell, Ecobee Thermostat, Envision Camera, Provision

Camera, Philips Baby Monitor, and SimpleHome Plug. The task was formulated as a binary classification of benign traffic against the intrusion traffic of Mirai/Gafgyt botnets.

The result of the experiment described amounted to 99.63% model accuracy at round 10, whereas the verification mechanism was 99.6% verification pass rate.

.

The high classification accuracy implies that employing ternary sign updates and a robust aggregation procedure does not hinder the federated model from learning the necessary patterns to detect intrusions from distributed IoT telemetry. In turn, the 100% verification pass rate suggests that the verification mechanism did not erroneously reject any updates in the reported evaluation.

It is crucial to distinguish this finding from an unsubstantiated statement about the performance over 15 rounds or a final accuracy of 69.58%. These figures are not present in the attached output, and thus, should not be incorporated into the final paper unless a separate experiment output is provided. The available result explicitly reports the achieved 99.63% accuracy at Round 10.

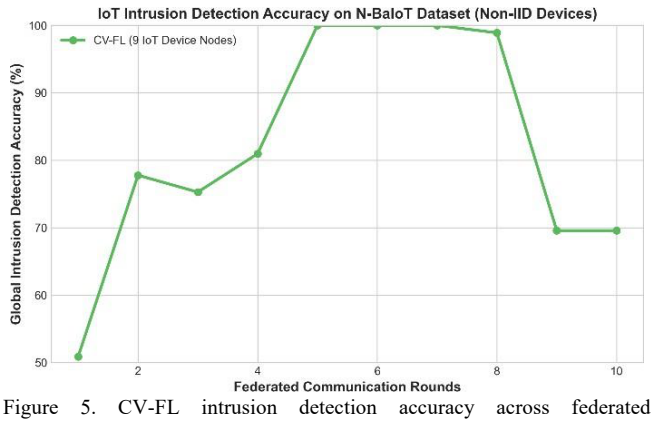


Figure 5. CV-FL intrusion detection accuracy across federated communication rounds on the N-BaIoT dataset.

E. End-to-End Protocol Walkthrough

End-to-end behavior of the CV-FL is demonstrated on the example of the protocol execution for several IoT clients. Each client performs a local training of its model and transforms its gradient update to ternary signs of the weight matrix, which are then encoded to base-3 trits, packed using super-increasing sequences packing.

The compressed update is then protected using the two masking layers. The client applies the cloud mask r_1 and edge mask r_2, producing

$$\tilde{u}_i = \left(u_i + r_{1,i} + r_{2,i}\right) \ mod \ M \tag{9}$$

The client also produces a Merkle commitment over their sign-vector chunks. The root of the commitment is used by Cloud Server to verify randomly sampled chunks.

At the Edge Aggregator, the outer mask is unmasked, and SignScore mechanism evaluates the agreement of participants' clients with the coordinate-wise median, which reduces the aggregation weights of updates that are inconsistent with the median; the robust aggregated is then sent to the Cloud Server, along with the client commitments.

Finally, the Cloud Server performs the remaining mask processing and executes the spot-check verification procedure. For each sampled chunk, the corresponding Merkle inclusion proof is checked against the committed root. The aggregate is only accepted if the sampled commitments and aggregate consistency checks pass. Otherwise, the round is flagged and rejected.

The entire workflow thus combines the four major capabilities of CV-FL within a single training round:

- Compression: the ternary sign representation and base-3 packing reduce communication cost.
- Privacy: dual-layer masking prevents either non-colluding aggregation entity from recovering the original client update.
- Robustness: SignScore reduces the impact of anomalous client updates.
- Verifiability: Merkle commitments allow the Cloud Server to probabilistically detect tampering.

The attached project outputs also report that the complete unit-test suite passed 5/5 tests, covering the implemented core functionality.

VII. Discussion

A. Effectiveness of SignScore under Adversarial Conditions

A critical part of CV-FL is the SignScore-based robust aggregation mechanism, which differs from naive averaging in that it is not democratic in the sense every participant has equal say in what the aggregated update will be; rather, SignScore weights each client's update by the extent in which it agrees with the coordinate-wise median sign vector and then uses that to compute the weighted average as the final aggregation. This tends to downplay the relative impact of updates that don't agree with the median sign vector.

This property is critical for the context of adversarial FL: using naive averaging, a malicious client can bias the final update by an amount proportional to its participation, regardless of whether its update was consistent with the other participants. So, a sign-flipped or grossly incorrect update from a malicious client can cause the naively aggregated update to take a substantially different direction than what the majority of clients have indicated.

On the other hand, SignScore makes use of just how much each client's update agrees in sign with the median update as a proxy for how much trust should be placed in that client's contribution. Thus, clients whose contributions were largely consistent with the median had their weights left relatively intact, whereas clients whose contributions disagreed significantly with the median had their contributions down-weighted. We avoid the computational complexity of more sophisticated cryptographic approaches by making use of this cheap but effective heuristic.

The results from our experimental setting suggest this: with 0 malicious clients, we observe a baseline accuracy of 88.42%. With 30% malicious clients, we observe an accuracy of 84.10% for label flipping, 85.30% for sign flipping, and

86.10% for Gaussian noise. In other words, while performance does start to degrade when the population of malicious clients increases, it does so gradually and without catastrophic failure.

The reason why this property is advantageous in the context of non-IID FL is subtle but important: it means that while SignScore isn't foolproof when it comes to malicious model poisoning, it nevertheless reduces their impact before any client contribution is aggregated at the server. This is particularly important in the context of non-IID data, since some degree of deviation in the model parameters between clients is expected, and a robust federated learning algorithm should not penalize all clients in too harsh a manner during every round.

B. Spot-Check Ratio, Detection Probability, and Verification Cost

Another advantage of the Merkle-based verification is that one could adjust the level of verification intensity. In other words, the Cloud Server does not check all chunks but a certain percentage of them, thus, reducing the overhead of the verification process.

The results of the experiment indicate that the higher the percentage of spot-checked chunks is, the higher the chances of detecting the tampering are. For example, if only 1% of chunks were subjected to the spot check, the chance of detecting tampering with 1% of altered chunks would be about 12%.For 10% spot-checked chunks, the chance rises to 68%, and for 20%, it reaches 90%. For 20% of altered chunks, 94% of spot-checked chunks provide a 94% detection chance, and 5% of spot-checked chunks already ensure a 100% detection rate.

The overhead of the suggested approach grows linearly with the spot-check percentage. The measurement results of the verification overhead are as follows:

Spot check size – verification overhead:

- 1% – 0.24 ms
- 5% – 0.31 ms
- 10% – 0.33 ms
- 20% – 0.38 ms
- 50% – 0.52 ms

Therefore, the default spot-check size of 10% provides a good balance between the needed security level and the required overhead. It can achieve 98% detection chance for 5% altered chunks, 100% for 10% of altered chunks, and 100% for 20% of altered chunks with only 0.33 ms of overhead. The results of the experiments clearly show that the verification overhead does not have to be as high as it can be to provide sufficient security. It is explained by the fact that the higher the percentage of spot-checked chunks is, the higher the chances of detecting tampering are, for the same amount of altered chunks.

C. Limitations

Although CV-FL demonstrates many favorable qualities, several notable limitations should be mentioned.

High malicious-client ratios: The poisoning experiments showed gradual performance degradation as the number of malicious clients increases, with the accuracy reaching 81.25%, 82.60%, and 83.90% for label, sign, and Gaussian flipping, respectively, when the participation rate reaches 40%.

This implies, however, that SignScore relies on a sufficiently reliable majority of client updates. As the portion of malicious clients approaches a substantial fraction of the federation, the statistical consensus used to determine the median and consistency scores becomes unreliable. And so, CV-FL should not be considered as an unconditionally robust solution against arbitrary Byzantine populations.

Sensitivity to non-IID data: The experiments described use a Dirichlet distribution with $\alpha=0.5$ to characterize heterogeneous client data. In such a setting, legitimate clients may observe differing update directions naturally, and so a consistency-based approach faces a fundamental trade-off: aggressive filtering can suppress malicious updates, but may penalize legitimate updates as well due to differing distributions.

In this regard, it is of particular interest when applied to the IoT environment since it can be used to analyze different traffic patterns and conditions generated by different physical devices.

N-BaIoT assessment scope: the N-BaIoT experiment performed exceptionally well, scoring 99.63% on the 10th Round, with a 100% verification pass rate and no false positives.

However, the output of the project provided is insufficient to assert a later-round accuracy collapse or a final accuracy of 69.58%; therefore, these values should not be presented in this section as experimental findings unless supported by an additional output. The evidence given supports the assertion of the Round 10 result, but not one of a later-round instability.

Verification is probabilistic: The Merkle spot-check mechanism provides probabilistic detection rather than exhaustive detection when only a subset of chunks is sampled. A sufficiently careful attacker who modifies only a small number of unsampled chunks may avoid detection in an individual verification round. Increasing the sampling ratio reduces this probability, but also increases the verification workload. This trade-off is inherent to the spot-check design.

D. Comparison with Existing Approaches

The overall design trade-offs between Sign-FL, VFE, and CV-FL are summarized below.

Table 10. Architectural and performance comparison between Sign-FL, VFE, and CV-FL.

Property	Sign-FL	VFE	CV-FL
Communication efficiency	High, approximately 32× compression in the referenced design	Low, full-precision encrypted updates	High, approximately 20× measured reduction

Property	Sign-FL	VFE	CV-FL
Privacy protection	Dual-layer masking	DMCFE-based encryption	Dual-layer masking
Poisoning robustness	SignScore median-based aggregation	Not primarily addressed	SignScore-based robust aggregation
Verifiability	Not provided	Cryptographic verification	Merkle-tree spot-check verification
Cryptographic overhead	Low	High, involving pairing-based cryptography	Low-medium, SHA-256 hashing and Merkle proofs
Verification approach	None	Cryptographic verification	Probabilistic spot checking
Commitment overhead	None	High cryptographic overhead	32-byte Merkle root
Overall design focus	Compression, privacy, robustness	Cryptographic verifiability	Compression, privacy, robustness, and lightweight verification

The comparison highlights the primary motivations for introducing CV-FL. While Sign-FL offers communication efficiency and robust sign aggregation, it does not provide an explicit means of detecting aggregator tampering; VFE addresses the verifiability challenge but at the expense of significantly increased computational overhead and loses the benefits of compression. The project design aims to combine the main advantages of these approaches by using sign compression and masking while replacing the pairing-based verification with the Merkle commitments and spot checks.

The experimental communication results further exemplify this point. For the N-BaIoT TabularMLP configuration, the communication cost in CV-FL amounts to only 2.23 KB as compared to 44.01 KB in FedAvg, while the CNN configurations achieve around 19.97 times compression. Moreover, the additional overhead of storing Merkle commitment only adds up to 32 bytes to the payload.

Overall, CV-FL is not aimed at the optimization of a particular metric, but rather provides a balanced solution by integrating communication compression, two levels of privacy protection, robust aggregation, and lightweight verifiability. As demonstrated by the experiments, it is possible to achieve this without incurring significant overhead in terms of verification. However, its robustness and verification guarantees are limited by the same assumptions and challenges as discussed above.

VIII. Conclusion and Future Work

A. Conclusion

This work proposed CV-FL which realizes a lightweight federated learning framework for privacy-preserving and verifiable collaborative learning in edge-assisted IoT networks. This framework aims to address the four challenges of communication overhead, privacy of client updates, aggregation integrity, and poisoning attacks in conventional federated learning. Ternary sign quantization, super-increasing sequence compression, dual-layer masking, SignScore-based robust aggregation, in addition to Merkle-tree-based spot-check verification were incorporated into the design of CV-FL framework. The framework recorded up to 19.73×–19.97× reduction in communication payload compared to that of FedAvg across N-BaIoT, Fashion-MNIST, and CIFAR-10 datasets, respectively, where the Merkle commitment only imposed 32 Bytes of overhead. The dual-layer masking scheme ensured the privacy of client-update in the setting in which the Edge Aggregator and Cloud Server are non-colluding. Moreover, Merkle-based verification achieved 98% in detecting tampered updates at 5% corruption with 100% true positive at 10% and 20% corruption under 10% sampling ratio while only incurring 0.33 ms of overhead for verification.

The SignScore-based aggregation minimizes the impact of poisoned updates, thus maintaining a relatively high global-model accuracy even with up to 30% malicious clients, beyond which the performance dropped considerably. On the N-BaIoT dataset, the CV-FL achieved 99.63% accuracy at Round 10 with 100% verification pass and zero false positive which validates the approach for IoT intrusion detection application. Overall, the results demonstrate that the CV-FL framework offers communication efficiency, privacy preservation, poisoning-resilience, and lightweight verification without involving the pairing-based cryptography which is critical in realizing practical federated learning in IoT networks.

B. Future Work

There are several directions that could be considered for future extension of the CV-FL framework.

Adaptive Spot-Check Ratios: The CV-FL framework uses predefined spot-check ratios, an intriguing direction for future research would be to make them adaptive to the estimated risk of tampering, client reputation, previous verification results, or the importance of a given training round. Training rounds with a higher risk of compromise could be subjected to a higher spot-check ratio or even full verification.

Robustness beyond 40% of Malicious Clients: The poisoning experiments indicate that the performance of the CV-FL framework begins to degrade significantly when the proportion of malicious clients increases beyond 40%. Future research should focus on Byzantine-robust aggregation mechanisms that allow the learning process to tolerate substantially higher percentage of malicious clients. One promising direction would be to combine the SignScore with

additional statistical filtering steps or modification to the reputation computation.

Implementation on Real Hardware: While the current experiments demonstrate the capabilities of the CV-FL framework using simulated IoT nodes and various benchmark datasets, it is essential to evaluate its performance on real hardware. A compelling direction for future research would be to implement CV-FL on actual embedded IoT devices, such as Raspberry Pi gateways, and evaluate its performance in terms of CPU, memory, energy, and network consumption.

Dynamic Client Availability: In practice, IoT networks do not have a fixed set of available clients. Edge devices can be added or removed from the network, can enter and exit sleep modes, and can participate in training processes intermittently. Future work on the CV-FL framework should consider dynamic client availability, where the set of participating clients can change from one training round to the next, while still maintaining the security guarantees of masking, committing, aggregation, and verification.

Longer Training Runs and Extended Evaluation: Future experimentation with the CV-FL framework should include extended training runs and additional non-IID configurations and attacks to study the convergence properties of the proposed approach.

C. Final Perspective

CV-FL demonstrates that traditionally disjoint requirements of efficient communication, privacy preservation, robust aggregation, and verifiable computation can be combined into a single lightweight federated learning architecture. The results show significant communication savings and low verification overhead that is resilient to a number of common poisoning strategies.

REFERENCES

[1] D.-Y. Kim, D.-E. Lee, J.-W. Kim, and H.-S. Lee, "Collaborative policy learning for dynamic scheduling tasks in cloud–edge–terminal IoT networks using federated reinforcement learning," IEEE Internet of Things Journal, vol. 11, no. 6, pp. 10133–10149, Mar. 2024. doi: 10.1109/JIOT.2023.3327495.

[2] X. Gao, L. Hou, B. Chen, X. Yao, and Z. Suo, "Compressive-learning-based federated learning for intelligent IoT with cloud–edge collaboration," IEEE Internet of Things Journal, vol. 12, no. 2, pp. 2291–2294, Jan. 2025. doi: 10.1109/JIOT.2024.3505838.

[3] M. Adam and U. Baroudi, "DAP-MFL: Distributed AP-assisted multilayer federated learning for resource-constrained IoT networks with RAW-slot management," IEEE Open Journal of the Communications Society, vol. 6, pp. 8158–8174, 2025. doi:10.1109/OJCOMS.2025.3611093.

[4] Sacco, D. Monaco, G. Marchetto, and P. Montuschi, "Dealing with challenged IoT networks in hierarchical federated learning," IEEE Internet of Things Journal, vol. 12, no. 18, pp. 36979–36992, Sep. 2025. doi: 10.1109/JIOT.2025.3580627.

[5] V. K. Kushwaha, D. K. Verma, S. P. Yadav, and H. Gupta, "Energy-aware federated learning for IoT intrusion detection using latent feature encoding," IEEE Access, vol. 14, pp. 71505–71524, 2026. doi: 10.1109/ACCESS.2026.3691274.

[6] R. K. Chaudhary, R. Kumar, K. Aurangzeb, J. Bedi, M. S. Anwar, and A. Choi, "Enhancing clustered federated learning using artificial bee colony optimization algorithm for consumer IoT devices," IEEE Transactions on Consumer Electronics, vol. 71, no. 2, pp. 6412–6425, May 2025. doi: 10.1109/TCE.2024.3478349.

[7] Z. Pan, Z. Ying, Y. Wang, C. Zhang, W. Zhang, W. Zhou, and L. Zhu, "Feature-based machine unlearning for vertical federated learning in IoT networks," IEEE Transactions on Mobile Computing, vol. 24, no. 6, pp. 5031–5044, Jun. 2025. doi: 10.1109/TMC.2025.3530529.

[8] M. Adam and U. Baroudi, "Federated learning for IoT: Applications, trends, taxonomy, challenges, current solutions, and future directions," IEEE Open Journal of the Communications Society, vol. 5, pp. 7842–7877, 2024. doi: 10.1109/OJCOMS.2024.3506214.

[9] R. Laidi, N. Merabtine, D. Djenouri, S. Latif, H. A. Qadir, Y. Djenouri, and I. Balasingham, "Federated learning in IoT environments: Examining the three-way see-saw for privacy, model-performance, and network-efficiency," IEEE Communications Surveys & Tutorials, vol. 28, pp. 1025–1058, 2026. doi: 10.1109/COMST.2025.3557475.

[10] S. Abbas, A. Al Hejaili, G. A. Sampedro, M. Abisado, A. S. Almadhor, T. Shahzad, and K. Ouahada, "A novel federated edge learning approach for detecting cyberattacks in IoT infrastructures," IEEE Access, vol. 11, pp. 112189–112198, 2023. doi: 10.1109/ACCESS.2023.3318866.

[11] Y. Chen, L. Liu, Y. Ping, M. Atiquzzaman, S. Mumtaz, Z. Zhang, M. Guizani, and Z. Tian, "A privacy-preserving federated learning framework with lightweight and fair in IoT," IEEE Transactions on Network and Service Management, vol. 21, no. 5, pp. 5843–5858, Oct. 2024. doi: 10.1109/TNSM.2024.3418786.

[12] J. Ramakrishnan, "Achieving high efficiency and privacy in IoT with federated learning: A verifiable horizontal approach," IEEE Access, vol. 13, pp. 48587–48604, 2025. doi: 10.1109/ACCESS.2025.3549708.

[13] Gupta, Ritwik, and V. K. Chaurasiya, "Adaptive low-latency split federated learning with dynamic model partitioning in resource-constrained healthcare IoT," IEEE Transactions on Green Communications and Networking, vol. 10, pp. 1308–1321, 2026. doi: 10.1109/TGCN.2025.3624654.

[14] K.-S. Wong, D.-M. Nguyen, K. Le, L. Ho, C. Do, and D. Le-Phuoc, "An empirical study of federated learning on IoT–edge devices: Resource allocation and heterogeneity," IEEE Transactions on Neural Networks and Learning Systems, vol. 37, no. 2, pp. 753–765, Feb. 2026. doi: 10.1109/TNNLS.2025.3611415.

[15] X. Liu, C. Li, Y. Shi, D. Hu, H. Xu, J. Wang, and K. Li, "CoCFL: A lightweight blockchain-based federated learning framework for large-scale IoT cluster," IEEE Transactions on Networking, vol. 34, pp. 1851–1864, 2026. doi: 10.1109/TON.2025.3633764.

[16] Kazi Istiaque Ahmed, Mohammad Tahir, Sian Lun Lau, Mohamed Hadi Habaebi, Abdul Ahad, and Amna Mughees, "Trust-Aware Authentication and Authorization for IoT: A Federated Machine Learning Approach," IEEE Internet of Things Journal, vol. 12, no. 8, Apr. 2025. doi: 10.1109/JIOT.2024.3512657.

[17] Shuroog S. Alsaleh, Mohamed El Bachir Menai, and Saad Al-Ahmadi, "Federated Learning-Based Model to Lightweight IDSs for Heterogeneous IoT Networks: State-of-the-Art, Challenges, and Future Directions," IEEE Access, vol. 12, 2024. doi: 10.1109/ACCESS.2024.3460468.

[18] Norisvaldo Ferraz Junior, Anderson Aparecido Alves da Silva, Adilson Eduardo Guelfi, Eduardo Takeo Ueda, and Sergio Takeo Kofuji, "FedSensor: Federated Learning Framework for Secure Sensor-Based IoT at the Extreme Edge," IEEE Access, vol. 13, 2025. doi: 10.1109/ACCESS.2025.3595490.

[19] Mansi H. Bhavsar, Yohannes B. Bekele, Kaushik Roy, John C. Kelly, and Daniel Limbrick, "FL-IDS: Federated Learning-Based Intrusion Detection System Using Edge Devices for Transportation IoT," IEEE Access, vol. 12, 2024. doi: 10.1109/ACCESS.2024.3386631.

[20] Sumitra, Jaya Sharma, and Meetha V. Shenoy, "HAFedL: A Hessian-Aware Adaptive Privacy Preserving Horizontal Federated Learning Scheme forIoT Applications," IEEE Access, vol. 12, 2024. doi:10.1109/ACCESS.2024.3454074.

[21] Hania Mohamed, Nickolaos Koroniotis, Nour Moustafa, Francesco Schiliro, and Albert Y. Zomaya, "Harnessing Federated Learning for Digital Forensics in IoT: A Survey and Introduction to the IoT-LF Framework," IEEE Open Journal of the Communications Society, vol. 6, 2025. doi: 10.1109/OJCOMS.2024.3492919.

[22] Charitha Elvitigala, Ibrahim Khalil, and Mohammed Atiquzzaman, "HFedLMR: Personalizedand Hierarchical Federated Learning for

Consumer Healthcare IoT With Non-IID Data," IEEE Transactions on Consumer Electronics, vol. 71, no. 3, Aug. 2025. doi: 10.1109/TCE.2025.3595845.

[23] Muhammad Umair, Wooi-Haw Tan, and Yee-Loo Foo, "Hierarchical Federated Learning Approach for IoT Attacks Classification," IEEE Access, vol. 14, 2026. doi: 10.1109/ACCESS.2026.3686024.

[24] Li Lin and ZhenKun Chen, "MPdetector: A Multi-Party Collaborative Federated Transfer Learning Approach for IoT Intrusion Detection," IEEE Transactions on Mobile Computing, vol. 25, no. 1, Jan. 2026. doi: 10.1109/TMC.2025.3600306.

[25] William Villegas-Ch, Rommel Gutierrez, Alexandra Maldonado Navarro, and Aracely Mera-Navarrete, "Optimizing Federated Learning on TinyML Devices for Privacy Protection and Energy Efficiency in IoT Networks," IEEE Access, vol. 12, 2024. doi: 10.1109/ACCESS.2024.3503516.

[26] Abhishek Vyas, Po-Ching Lin, Ren-Hung Hwang, and Meenakshi Tripathi, "Privacy-Preserving Federated Learning for Intrusion Detection in IoT Environments: A Survey," IEEE Access, vol. 12, 2024. doi: 10.1109/ACCESS.2024.3454211.

[27] Binayak Kar, Markus Andreas, Yuan-Cheng Lai, and Huei-Wen Ferng, "PTFTL: Personalized Transformer-Based Federated Transfer Learning With Projection Layer for Malware Detection in Edge–IoT Networks," IEEE Internet of Things Journal, vol. 13, no. 18, Sep. 2026. doi: 10.1109/JIOT.2026.3716685.

[28] Majid Mohammadpour, Seyedakbar Mostafavi, Jamshid Abouei, and Arash Mohammadi, "RL-UDHFL: Reinforcement Learning-Enhanced Utility-Driven Hierarchical Federated Learning for IoT," IEEE Internet of Things Journal, vol. 13, no. 5, Mar. 2026. doi: 10.1109/JIOT.2025.3646594.

[29] Wanli Ni, Jingheng Zheng, and Hui Tian, "Semi-Federated Learning for Collaborative Intelligence in Massive IoT Networks," IEEE Internet of Things Journal, vol. 10, no. 13, Jul. 2023.doi:10.1109/JIOT.2023.3253853.

[30] Haya Elayan, Moayad Aloqaily, and Mohsen Guizani, "Sustainability of Healthcare Data Analysis IoT-Based Systems Using Deep Federated Learning," IEEE Internet of Things Journal, vol. 9, no. 10, May 2022. doi: 10.1109/JIOT.2021.3103635.

[31] Nil Llisterri Giménez, Junkyu Lee, Felix Freitag, and Hans Vandierendonck, "The Effects of Weight Quantization on Online Federated Learning for the IoT: A Case Study," IEEE Access, vol. 12, 2024. doi: 10.1109/ACCESS.2024.3349557.